



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

DSP Lab - EE2801

Experiment-3

BY

Muthyala Koushik EE24BTECH11044

2026-02-22

Department of Electrical Engineering Indian Institute of Technology Hyderabad

Contents

1	Objective	1
2	Theory of Fixed Point Division using Newton–Raphson Method	2
3	Verilog and MATLAB Implementation	7
4	Observations	15
5	Conclusion	15

1 Objective

Fixed point division using numerical methods (Newton Raphson). Take a fraction like $15/23$, $10/79$ etc. Using Newton Raphson method, first compute $1/23$ and then multiply with 15. You need to do this in fixed point.

Do your work both in Matlab and Verilog & then verify Verilog results with MATLAB. Please include mathematical derivations also as part of your report.

You need to come up with a smart way of estimating initial guess. Do this exercise for array of fractions. Have a prime number in denominator.

Deliverables:

1. Verilog code
.v file
2. MATLAB code
.m file
3. PDF showing mathematical derivations for all your work

2 Theory of Fixed Point Division using Newton–Raphson Method

Introduction

1. Division is computationally expensive in digital systems.
2. Instead of performing direct division, numerical techniques are used to compute the reciprocal of the denominator and multiply it by the numerator.

$$Q = \frac{a}{b} = a \cdot \left(\frac{1}{b}\right)$$

Hence the problem reduces to computing the reciprocal efficiently.

Derivation of Newton Method from Taylor Series

Let $f(x)$ be differentiable and α be its root:

$$f(\alpha) = 0$$

Using Taylor expansion around x_n :

$$f(x) = f(x_n) + (x - x_n)f'(x_n) + \frac{(x - x_n)^2}{2}f''(\xi)$$

Neglecting higher order terms:

$$f(x) \approx f(x_n) + (x - x_n)f'(x_n)$$

Setting $f(x) = 0$:

$$0 = f(x_n) + (x - x_n)f'(x_n)$$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

This is the Newton–Raphson iteration formula.

Newton–Raphson Reciprocal Computation

To compute reciprocal of d :

$$x = \frac{1}{d}$$

Define:

$$f(x) = \frac{1}{x} - d$$

$$f'(x) = -\frac{1}{x^2}$$

Applying Newton iteration:

$$x_{n+1} = x_n - \frac{\frac{1}{x_n} - d}{-\frac{1}{x_n^2}}$$

$$x_{n+1} = x_n + x_n(1 - dx_n)$$

$$x_{n+1} = x_n(2 - dx_n)$$

Quadratic Error Analysis

$$\text{error} : \epsilon_n = 1 - dx_n$$

Next iteration:

$$\epsilon_{n+1} = 1 - dx_{n+1}$$

$$= 1 - d[x_n(2 - dx_n)]$$

$$= 1 - 2dx_n + d^2x_n^2$$

$$= (1 - dx_n)^2$$

$$\epsilon_{n+1} = (\epsilon_n)^2$$

Thus Newton method converges quadratically.

Fixed Point Representation

Q16 format is used:

$$\text{Real value} = \frac{\text{Integer value}}{2^{16}}$$

Multiplication produces Q32 result, hence right shift by 16 restores Q16 scaling.

Normalization of Denominator

For faster convergence:

$$0.5 \leq d \leq 1$$

This bounds reciprocal between 1 and 2.

Normalization is performed using bit shifting.

Chebyshev Equioscillation for Initial Approximation

For a good initial estimate of reciprocal, we approximate:

$$\frac{1}{d} \approx a + bd$$

To obtain minimum maximum error over interval $[0.5, 1]$, Chebyshev equioscillation principle is used.

Define approximation error:

$$E(d) = \frac{1}{d} - (a + bd)$$

Multiplying by d :

$$F(d) = 1 - d(a + bd) = 1 - ad - bd^2$$

Extrema occur when:

$$F'(d) = 0 \Rightarrow -a - 2bd = 0$$

$$d_m = -\frac{a}{2b}$$

Equioscillation condition:

$$F(0.5) = -F(d_m) = F(1)$$

Coefficient Calculation

Evaluating endpoints:

$$F(1) = 1 - a - b$$

$$F(0.5) = 1 - \frac{a}{2} - \frac{b}{4}$$

Equating:

$$1 - a - b = 1 - \frac{a}{2} - \frac{b}{4} \Rightarrow a = -\frac{3b}{2}$$

Using midpoint condition and solving:

$$b = -\frac{32}{17}, \quad a = \frac{48}{17}$$

Thus optimal linear minimax approximation:

$$x_0 = \frac{48}{17} - \frac{32}{17}d$$

A simpler hardware-friendly approximation is:

$$x_0 = 4(\sqrt{3} - 1) - 2d$$

This provides a good initial guess with small error and fast Newton convergence.

Complete Algorithm

1. Convert numerator and denominator to Q16 format.
2. Normalize denominator.
3. Compute initial estimate x_0 .
4. Perform Newton iterations:

$$x_{n+1} = x_n(2 - dx_n)$$

5. Undo normalization.
6. Multiply with numerator.

Final result:

$$\frac{a}{b} \approx ax_n$$

3 Verilog and MATLAB Implementation

Overview

Fixed-point division is implemented using the Newton–Raphson method. Instead of direct division, the reciprocal of the denominator is computed and multiplied with the numerator.

$$\frac{a}{b} = a \cdot \frac{1}{b}$$

All computations are performed in Q16 fixed-point format. The design is verified by comparing Verilog results with MATLAB results.

LUT Generation for Initial Estimate

To ensure fast convergence of Newton–Raphson method, a good initial estimate of reciprocal is required. A lookup table (LUT) is generated for values of denominator in the range:

$$0.5 \leq d \leq 1$$

The initial estimate used is:

$$x_0 = 4(\sqrt{3} - 1) - 2d$$

The interval is divided into 64 points and stored in `lut.txt`. These values are later used by the division module.

Listing 1: LUT Generator Code

```

1 module lut_generator;
2
3 integer file;
4 integer i;
5 real temp_d;
6 real temp_x;
7 integer val;
8
9 initial begin
10
11     file = $fopen("lut.txt", "w");
12
13     for(i=0; i<64; i=i+1) begin
14         temp_d = 0.5 + (i*(0.5/64.0));
15         temp_x = 4.0*($sqrt(3.0)-1.0) - 2.0*temp_d;

```

```

16         val = $rtoi(temp_x * (2.0**16));
17
18         $fwrite(file,"%d\n",val);
19     end
20
21     $fclose(file);
22     $finish;
23 end
24
25 endmodule

```

Division Module

The division module performs fixed-point division using Newton–Raphson iteration.
Main steps:

- Convert inputs to Q16 format
- Normalize denominator to range 0.5 to 1
- Obtain initial reciprocal from LUT
- Perform three Newton iterations
- Undo normalization
- Multiply with numerator to obtain final result

Newton iteration used:

$$x_{n+1} = x_n(2 - dx_n)$$

The final output is obtained in Q16 format.

Listing 2: Division Module

```

1 module division (
2     input signed [32:0] a,
3     input signed [32:0] b,
4     input signed [32:0] lut [0:63],
5     output reg signed [32:0] div
6 );
7
8 // Convert to Q16

```

```

9  wire signed [32:0] q_a;
10 wire signed [32:0] q_b;
11
12 assign q_a = a <<< 16;
13 assign q_b = b <<< 16;
14
15 // Normalization
16 reg signed [32:0] d;
17 reg signed [6:0] shift;
18
19 always @(*) begin
20     d = q_b;
21     shift = 0;
22
23     while (d >= (33'sd1 <<< 16)) begin
24         d = d >>> 1;
25         shift = shift + 1;
26     end
27
28     while (d < (33'sd1 <<< 15)) begin
29         d = d <<< 1;
30         shift = shift - 1;
31     end
32 end
33
34 // Index for LUT
35 wire [5:0] index;
36 assign index = (d - 33'sd32768) >>> 9;
37
38 // Initial estimate
39 reg signed [32:0] x0;
40 always @(*) begin
41     x0 = lut[index];
42 end
43
44 // Newton Raphson
45 reg signed [32:0] x;
46 reg signed [65:0] t1;
47 reg signed [65:0] t2;
48 reg signed [65:0] final_mult;

```

```

49
50 integer j;
51
52
53 always @(*) begin
54     // division by zero protection
55     if (b == 0) begin
56         div = 0;
57     end
58     else begin
59         x = x0;
60         for(j=0;j<3;j=j+1) begin
61             t1 = d * x;
62             t1 = t1 >>> 16;
63
64             t2 = (33'sd2 <<< 16) - t1;
65
66             t2 = x * t2;
67             x = t2 >>> 16;
68         end
69
70         // undo normalization
71         if (shift >= 0)
72             x = x >>> shift;
73         else
74             x = x <<< (-shift);
75
76         // final division
77         final_mult = q_a * x;
78         div = final_mult >>> 16;
79     end
80 end
81
82 endmodule
    
```

Testbench

The testbench performs the following tasks:

- Reads LUT values from file

- Applies different fractions with prime denominators
- Stores Verilog results in results.txt

These results are later verified using MATLAB.

Listing 3: Testbench

```
1 `timescale 1ns/1ps
2
3 module tb_division;
4
5 reg signed [32:0] a;
6 reg signed [32:0] b;
7 wire signed [32:0] div;
8
9 // LUT memory inside testbench
10 reg signed [32:0] lut_table [0:63];
11
12 integer file;
13 integer i;
14 integer val;
15 integer status; // required for $fscanf
16
17 // Instantiate division module
18 division uut (
19     .a(a),
20     .b(b),
21     .lut(lut_table),
22     .div(div)
23 );
24
25 initial begin
26     // read LUT
27     file = $fopen("lut.txt","r");
28
29     for(i=0;i<64;i=i+1) begin
30         status = $fscanf(file,"%d\n",val);
31         lut_table[i] = val;
32     end
33
34     $fclose(file);
```

```

35
36 // write results
37 file = $fopen("results.txt","w");
38
39 a=15; b=23; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
40 a=10; b=79; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
41 a=25; b=113; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
42 a=17; b=199; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
43 a=8; b=13; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
44 a=12; b=17; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
45 a=9; b=19; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
46 a=21; b=31; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
47 a=25; b=41; #20; $fwrite(file,"%0d %0d %0d\n",a,b,div);
48
49 $fclose(file);
50 $finish;
51 end
52
53 endmodule

```

MATLAB Verification

MATLAB is used to verify correctness of the Verilog implementation.

Steps performed:

- Read Verilog output file
- Perform same fixed-point Newton iterations
- Compute exact division
- Compare MATLAB, Verilog and exact values
- Store results in verification file

This confirms accuracy of hardware implementation.

Listing 4: MATLAB Verification Code

```

1 clc; clear;
2
3 Q = 2^16;
4

```

```
5 % Read LUT generated by Verilog
6 lut = load('lut.txt');
7
8 % read Verilog output file
9 data = load('results.txt');
10
11 a_vals = data(:,1);
12 b_vals = data(:,2);
13 verilog_q16 = data(:,3);
14
15 % open output file
16 fid = fopen('matlab_verification.txt','w');
17 fprintf(fid,'a b verilog_result matlab_result exact_value error_mv error_exact\
    n');
18
19 for k = 1:length(a_vals)
20
21     a = a_vals(k);
22     b = b_vals(k);
23
24     % Newton Q16
25     q_a = a * Q;
26     q_b = b * Q;
27
28     d = q_b;
29     shift = 0;
30
31     % normalization
32     while d >= (1*Q)
33         d = floor(d/2);
34         shift = shift + 1;
35     end
36
37     while d < (0.5*Q)
38         d = d * 2;
39         shift = shift - 1;
40     end
41
42     % initial guess
43     index = floor((d - 32768) / 512); % same as >>>9 in Verilog
```

```

44
45 % Safety clamp
46 if index < 0
47     index = 0;
48 elseif index > 63
49     index = 63;
50 end
51
52 x = lut(index + 1);
53
54 % Newton iterations
55 for i = 1:3
56     t1 = floor((d * x)/Q);
57     t2 = 2*Q - t1;
58     x = floor((x * t2)/Q);
59 end
60
61 % undo normalization
62 if shift >= 0
63     x = floor(x / (2^shift));
64 else
65     x = x * (2^(-shift));
66 end
67
68 % final multiply
69 matlab_q16 = floor((q_a * x)/Q);
70
71 % convert to real
72 verilog_real = verilog_q16(k)/Q;
73 matlab_real = matlab_q16/Q;
74 exact = a/b;
75 error = abs(matlab_real - verilog_real);
76 error_exact = abs(exact - verilog_real);
77
78 % write to file
79 fprintf(fid, '%d %d %.10f %.10f %.10f %.10e %.10e\n', a, b, verilog_real,
80     matlab_real, exact, error, error_exact);
81 end
82

```

83 `fclose(fid);`

4 Observations

a	b	verilog_result	matlab_result	exact_value	error_mv	error_exact
15	23	0.6520843506	0.6520843506	0.6521739130	0.0000000000e+00	8.9562457541e-05
10	79	0.1264953613	0.1264953613	0.1265822785	0.0000000000e+00	8.6917152888e-05
25	113	0.2208709717	0.2208709717	0.2212389381	0.0000000000e+00	3.6796637341e-04
17	199	0.0853424072	0.0853424072	0.0854271357	0.0000000000e+00	8.4728451829e-05
8	13	0.6153564453	0.6153564453	0.6153846154	0.0000000000e+00	2.8170072115e-05
12	17	0.7058715820	0.7058715820	0.7058823529	0.0000000000e+00	1.0770909927e-05
9	19	0.4736480713	0.4736480713	0.4736842105	0.0000000000e+00	3.6139237253e-05
21	31	0.6773986816	0.6773986816	0.6774193548	0.0000000000e+00	2.0673198085e-05
25	41	0.6095886230	0.6095886230	0.6097560976	0.0000000000e+00	1.6747451410e-04

Figure 1: Comparison of Verilog, MATLAB and Exact Division Results

- Verilog and MATLAB results match exactly for all tested fractions.
- Error between MATLAB and Verilog output is zero.
- Small difference from exact value is due to Q16 fixed-point quantization.
- Maximum error observed is on the order of 10^{-4} .
- Three Newton–Raphson iterations provide sufficient accuracy.
- Lookup table improves convergence speed with a good initial estimate.

5 Conclusion

- Fixed-point division using Newton–Raphson method was successfully implemented.
- Reciprocal computation followed by multiplication gives accurate results.
- Verilog results were verified using MATLAB for multiple prime denominators.
- Very small error confirms correctness of Q16 implementation.
- Design uses only multipliers and shifts, making it hardware efficient.
- Suitable for FPGA and ASIC implementations.